

OLIST DATA PLATFORM

Full-Stack Data Platform: Lakehouse to Power BI Project Documentation

Technology	Databricks PySpark Delta Lake Unity Catalog Power BI GitHub Actions PySpark ML
Repository	github.com/goldenFlori/olist-data-platform
Dataset	Brazilian E-Commerce Public Dataset by Olist
Date	September 2026

Scope: medallion lakehouse, ingestion, data quality, dimensional modeling, orchestration, incremental loading, Power BI, performance, governance, CI, and ML scoring.

Contents

- 1. Executive summary
- 2. Architecture
- 3. Bronze ingestion and Auto Loader
- 4. Silver cleaning and data quality findings
- 5. Gold dimensional model
- 6. Orchestration, incremental loads, and the DQ gate
- 7. Power BI semantic model and report
- 8. Performance work
- 9. Bonus work
- 10. Challenges and lessons learned
- 11. Reproducibility / runbook
- 12. Submission summary

1. Executive summary

This project implements an end-to-end analytical data platform for the Brazilian E-Commerce Public Dataset by Olist. Nine raw CSV files are ingested into a governed Databricks lakehouse, cleaned and validated through a medallion architecture, modeled into a multi-fact Gold layer, orchestrated with explicit dependencies and incremental controls, optimized with Delta operations, and served to Power BI.

The base solution covers Bronze, Silver, Gold, orchestration, incremental processing, data quality, a Power BI semantic model, a multi-page report, and backend performance optimization. Four bonus items are complete and one is partially implemented relative to the exact wording of the exercise:

- Bonus 1 - Auto Loader / Structured Streaming: complete.
- Bonus 2 - Databricks Asset Bundle + GitHub Actions CI validation: complete.
- Bonus 3 - Unity Catalog lakehouse row-level security: complete.
- Bonus 4 - Import + DirectQuery reports: implemented, but the formal refresh/query/model-size comparison was not performed.
- Bonus 5 - ML delivery-delay output in Gold and Power BI: complete.

2. Architecture

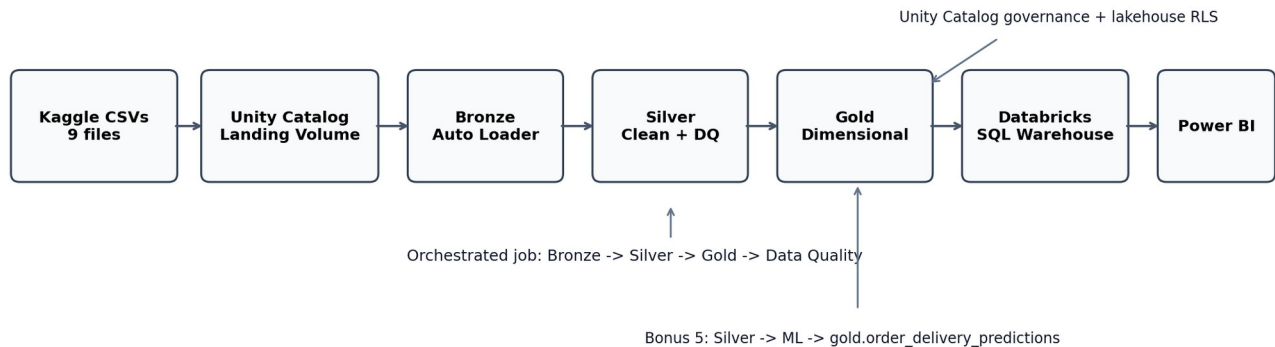


Figure 1. End-to-end Databricks lakehouse and reporting architecture.

The platform follows a medallion architecture with Unity Catalog governance and Delta Lake storage.

```

Kaggle CSVs
-> Unity Catalog landing volume
-> Bronze (Auto Loader + raw Delta + lineage metadata)
-> Silver (typed, cleaned, deduplicated, validated, quarantine)
-> Gold (facts, dimensions, aggregate)
-> Databricks SQL Warehouse
-> Power BI
  
```

```

Additional paths:
Silver -> ML feature engineering -> Random Forest -> Gold prediction table -> Power BI
Gold -> Unity Catalog entitlement mapping / row filters -> DirectQuery RLS demonstration
GitHub -> GitHub Actions -> databricks bundle validate
  
```

2.1 Why the layers are separated

Bronze is the source-aligned audit layer. It preserves the source values and adds lineage fields so every row can be traced to a source file and ingestion run.

Silver is the trusted layer. Type enforcement, profiling, deduplication, category translation, geolocation resolution, delivery-funnel derivation, and quarantine rules live here so business logic is not repeated in every downstream consumer.

Gold is the analytical serving layer. It exposes measures at their natural grain in a dimensional model, which keeps Power BI simple and prevents fan-out errors.

2.2 Storage and governance

All lakehouse tables use Delta Lake. Delta provides transactional writes, schema-aware operations, `MERGE` for idempotent incremental upserts, and support for optimization and data skipping.

Unity Catalog provides the three-level `catalog.schema.table` namespace, governed volumes, and the lakehouse security layer used for Bonus 3.

3. Bronze ingestion and Auto Loader

The project ingests all nine source files. Bronze adds three metadata fields to every record:

- `source_file`
- `ingestion_timestamp`
- `batch_id`

The final Bronze implementation uses Databricks Auto Loader (`cloudFiles`) rather than batch CSV reads. Each source has its own checkpoint under the control volume and runs with an `availableNow` trigger, giving streaming semantics while allowing a bounded ingestion run to complete.

Special handling:

- `order_reviews` uses multiline parsing because review comments can contain embedded newlines and quotes.
- `geolocation` is partitioned by `geolocation_state`, the selected physical partitioning strategy for the largest source.
- Explicit schemas are shared from the common schema notebook rather than inferred repeatedly.

3.1 Final Bronze validation

Bronze source	Rows
orders	99,441
order_items	112,650
order_payments	103,886
order_reviews	99,224
customers	99,441
sellers	3,095
products	32,951
geolocation	1,000,163
category_tr	71

All nine sources were validated with `metadata_nulls = 0`. Nine independent streaming checkpoint folders exist. `DESCRIBE DETAIL` confirmed Delta storage and partitioning of Bronze geolocation by `geolocation_state`.

The Bronze runner executes all nine ingestion notebooks and completed successfully before Silver, Gold, and the final DQ gate were rerun.

4. Silver cleaning and data quality findings

Every source is profiled for missing values, duplicates, and referential-integrity problems before the business cleaning rules are applied.

4.1 Delivery funnel

Silver derives the delivery funnel centrally so all consumers use the same definitions:

- approval time
- handling time
- shipping time
- total delivery time
- delivery variance against the estimated date

Orders that did not reach delivered status are retained. Delivery KPIs use only records where the relevant timestamps exist.

4.2 Key source issues and handling

Multiline review parsing. The review source initially exposed shifted values when read without multiline handling. Correct CSV parsing brought the Bronze count to the expected 99,224 source rows. Duplicate and invalid review records are handled downstream rather than silently dropped.

Review duplicates / invalids. Duplicate review identifiers are resolved using a deterministic rule. Rows that fail required review rules are quarantined.

Geolocation resolution. The roughly one-million-row geolocation source contains many rows per ZIP prefix and inconsistent city strings. Text normalization plus a deterministic per-ZIP resolution produces one reporting geography record per ZIP prefix.

Malformed seller city. A numeric/ZIP-like seller city value is repaired from valid ZIP-based geography rather than discarded.

Payments. Non-positive payment values are treated as invalid financial records and routed out of the trusted financial population.

Products. Products with missing descriptive attributes are retained so valid sales are not understated; missing category values are represented as an explicit uncategorized group.

4.3 Quarantine

Rows that fail business rules are routed to quarantine tables instead of being removed silently. This preserves auditability and allows the failure population to be reviewed independently of the trusted Silver model.

5. Gold dimensional model

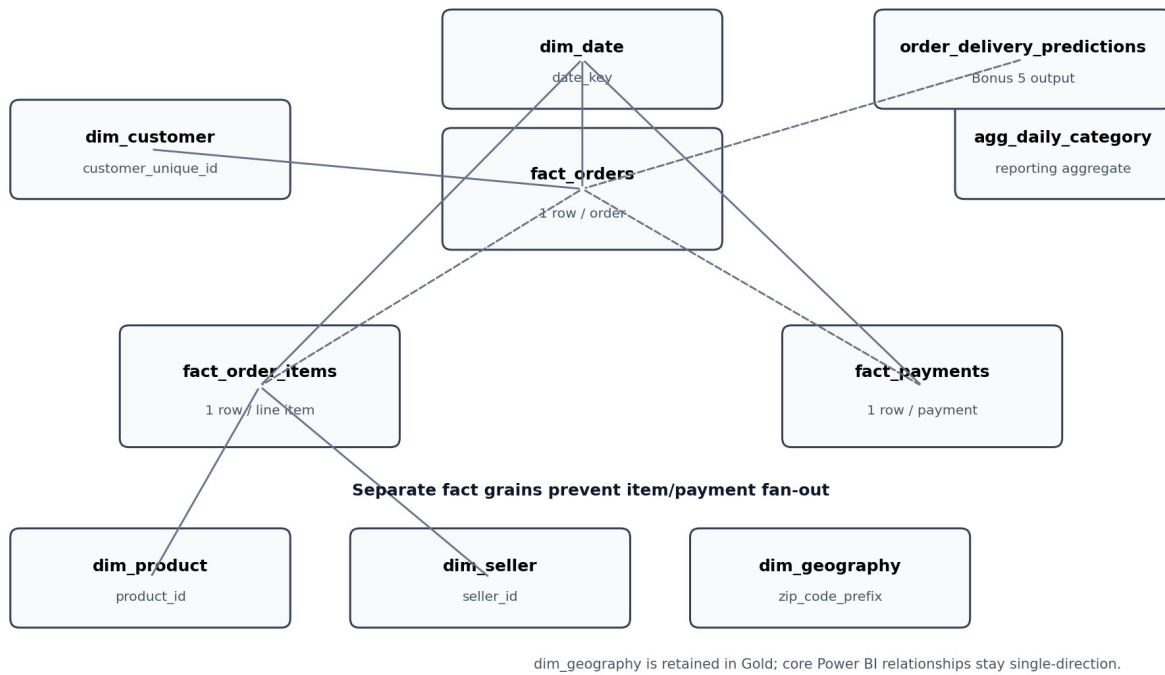


Figure 2. Core Gold multi-fact dimensional model and Bonus 5 prediction output.

The core Gold model contains three fact tables, five conformed dimensions, and one reporting aggregate. Bonus 5 adds a separate prediction table without changing the core facts.

5.1 Fact grains

Fact	Grain	Main content
fact_orders	One row per order	status, review score, delivery funnel metrics, on-time flag
fact_order_items	One row per line item	item price, freight value, product and seller keys
fact_payments	One row per payment	payment value, type, instalments

5.2 Dimensions

Dimension	Business key	Purpose
dim_customer	customer_unique_id	Represents the real customer across multiple orders
dim_product	product_id	English category and product attributes
dim_seller	seller_id	Seller location attributes
dim_date	date_key	Continuous pipeline-built date dimension
dim_geography	zip_code_prefix	Resolved ZIP-level geography

agg_daily_category is the reporting pre-aggregation required by the brief.

5.3 Customer grain

Olist source `customer_id` is order-specific. Returning people can receive a different `customer_id` on later orders. Therefore `customer_unique_id` is the business key for `dim_customer`, enabling correct distinct-customer and repeat-customer analysis.

5.4 Fan-out and multi-fact design

Items and payments are intentionally separate. An order can have several item rows and several payment rows. Joining them directly by `order_id` duplicates rows on both sides and inflates measures.

Measured impact:

- Correct item revenue: about BRL **13.6M**.
- Item revenue after fan-out: about BRL **14.2M**.
- Correct payment value: about BRL **16.0M**.
- Payment value after fan-out: about BRL **20.3M**.

Power BI therefore measures item revenue from `fact_order_items` and payment value from `fact_payments` independently.

6. Orchestration, incremental loads, and the DQ gate

6.1 Main orchestrated pipeline

The Databricks job definition is stored in `resources/jobs/olist_pipeline.yml` and declares four tasks:

```
run_bronze
  -> run_silver
    -> run_gold
      -> data_quality
```

The job definition references the orchestration notebooks as repository `.ipynb` files. Task dependencies prevent downstream layers from running before upstream completion.

6.2 Incremental processing

Incremental processing is driven by `order_purchase_timestamp`.

1. A historical load processes orders through the initial cutoff `2017-12-31`.
2. A control object stores the last successfully processed value as a watermark.
3. Incremental mode reads the watermark and selects only later records.
4. Delta `MERGE` updates matching business keys and inserts only new records.
5. The watermark advances after a successful write.

This makes reprocessing idempotent: rerunning an increment does not double-count the same business records.

6.3 Automated data quality gate

The final task fails the job if a required rule is breached. It covers:

- expected row counts / non-empty layer outputs
- source freshness based on latest order date
- uniqueness of dimension keys
- seven implemented fact-to-dimension referential-integrity checks
- range validation for price, freight, and review score

The final rerun after the Auto Loader migration passed all DQ checks.

7. Power BI semantic model and report

The primary Power BI solution uses Import mode against the Gold layer through a Databricks SQL Warehouse. Import was selected for the base report because the dataset is historical and small enough for fast VertiPaq interaction.

The base semantic model contains 15 core DAX measures covering:

- revenue, freight, average order value
- order count and distinct customer count
- on-time delivery rate
- average review score and low-review share
- repeat customer rate
- month-over-month and year-over-year comparisons
- payment totals used for reconciliation

Relationships use single-direction dimension-to-fact filtering, and `dim_date` is marked as the date table.

The original three-page Import model was approximately **24 MB** before the ML bonus table was added. No new formal cross-mode model-size benchmark is claimed after the bonus work.

7.1 Report pages

The Import report now contains four pages:

6. **Executive Overview** - headline KPIs, revenue trend, category contribution, and state breakdown.
7. **Delivery & Operations** - on-time delivery, funnel timings, delivery variance, and route/state analysis.
8. **Product & Seller Performance** - product/seller contribution, reviews, and customer behavior.
9. **Delivery Delay Prediction** - Bonus 5 test-set view showing actual versus predicted delivery delay and ML validation metrics.

The first three pages satisfy the base report requirement. Page 4 is an additional model-validation page.

8. Performance work

8.1 Backend optimization

A controlled revenue-by-date query was measured before and after Delta `OPTIMIZE` + `Z-ORDER`:

Stage	Time
Before	0.84 s
After	0.63 s
Improvement	25.3%

`Z-ORDER` improves **data skipping** by colocating related values so the Delta engine can avoid files that cannot match the query predicates. It is different from partition pruning. Bronze geolocation is physically partitioned by state; selected Gold fact columns use `Z-ORDER` for data skipping.

8.2 Semantic model

Only reporting-relevant Gold columns are loaded into the Power BI base model. The original Import model was approximately 24 MB before Bonus 5 added the prediction table.

8.3 Import / DirectQuery note

A DirectQuery variant was created, but no formal refresh-time, query-time, and model-size comparison was performed. Therefore the Import-vs-DirectQuery bonus is documented as partial relative to the exact exercise wording rather than overstated as a completed benchmark.

9. Bonus work

9.1 Bonus 1 - Auto Loader / Structured Streaming - Complete

All nine Bronze notebooks use Auto Loader instead of batch reads. The shared helper uses `cloudFiles`, explicit schemas, source-specific checkpoints, and `availableNow=True`. Each output remains Delta and retains ingestion metadata.

This migration was validated end to end: all nine source row counts matched, metadata was non-null, all checkpoint folders existed, geolocation partitioning remained correct, and the downstream Silver -> Gold -> DQ chain passed.

9.2 Bonus 2 - Databricks Asset Bundle + GitHub Actions CI - Complete

The project is packaged as a Databricks Asset Bundle:

```
databricks.yml
resources/jobs/olist_pipeline.yml
.github/workflows/validate-bundle.yml
```

`databricks.yml` includes the job resource files and defines the `dev` target. GitHub Actions checks out the repository, installs the Databricks CLI, authenticates through GitHub repository secrets, and runs:

```
databricks bundle validate --target dev
```

The final workflow run passed. The CI validates the bundle definition and notebook references; it does not deploy or execute the data pipeline because deployment was not required by the bonus wording.

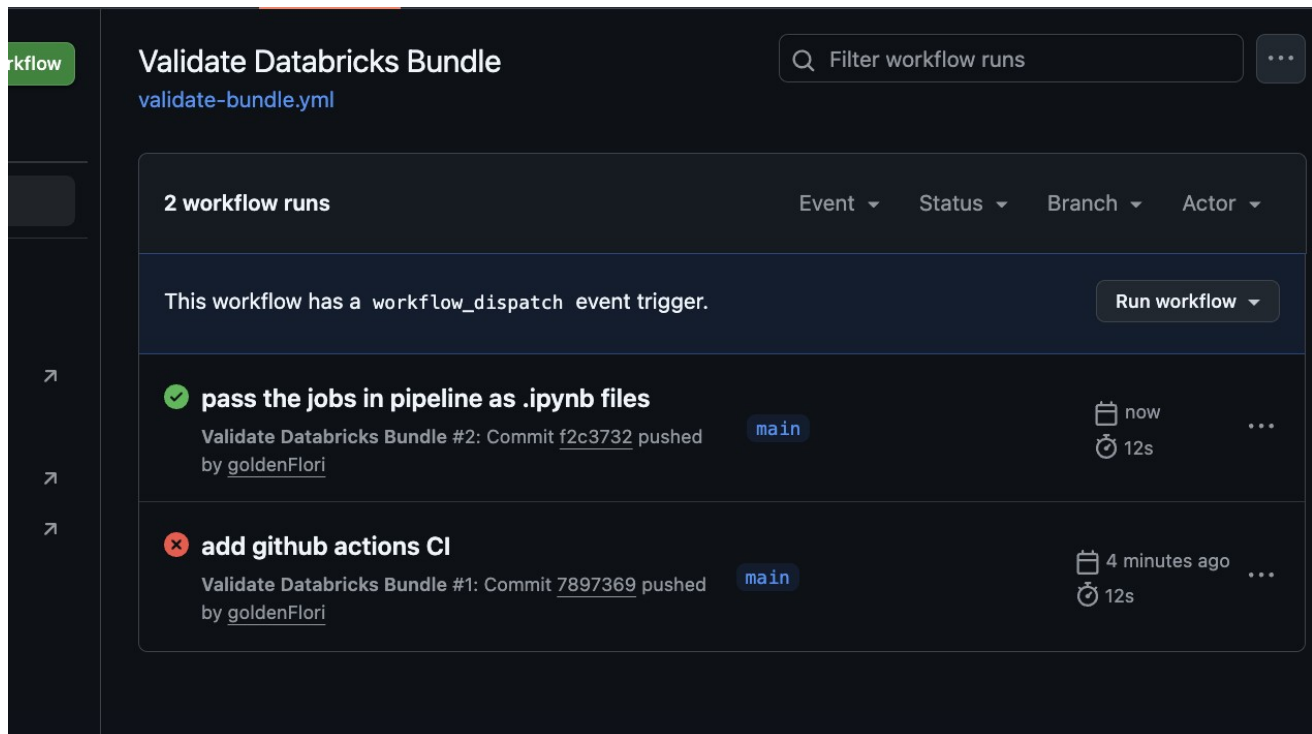


Figure 3. GitHub Actions validation of the Databricks Asset Bundle after correcting notebook file references.

9.3 Bonus 3 - Unity Catalog lakehouse RLS - Complete

Lakehouse row-level security is implemented in `04-gold/10_rls_lakehouse.ipynb`.

The implementation creates:

- `olist.control.seller_state_access` - user-to-state entitlement mapping
- `olist.control.seller_security_bridge` - seller-to-state bridge
- `olist.gold.seller_state_filter(state STRING)` - row-filter function for seller state
- `olist.gold.seller_id_filter(input_seller_id STRING)` - row-filter function for seller IDs

Row filters are applied to:

- `olist.gold.dim_seller` on `seller_state`
- `olist.gold.fact_order_items` on `seller_id`

Validation for the assigned state SP produced:

- baseline sellers: 3,095
- visible sellers after RLS: 1,849
- baseline order-item rows: 112,650
- visible order-item rows after RLS: 80,342
- visible seller states: 1
- rows visible outside assigned state: 0

This demonstrates that the restriction is enforced in Unity Catalog rather than only inside Power BI.

9.4 Bonus 4 - Import + DirectQuery - Partial relative to the full wording

Two Power BI variants were created:

```
powerbi/olist_report_import.pbix
powerbi/olist_report_directquery.pbix
```

The DirectQuery variant was used to demonstrate source-backed behavior and Unity Catalog RLS for seller-scoped data. The implementation proves that the report can query the governed Gold layer without importing the protected fact rows.

The exercise also asks for a formal comparison of refresh time, query time, and model size. That benchmark was intentionally **not performed**, so this bonus is documented as partially implemented rather than claimed as fully complete.

The DirectQuery RLS demonstration is also not a strict like-for-like performance population against the full Import model because the lakehouse policy restricts the seller-scoped DirectQuery data to SP.

9.5 Bonus 5 - ML delivery-delay output - Complete

The ML work is isolated from the core pipeline in `04-gold/11_delivery_delay_predictions.ipynb`. It reads trusted Silver data and writes a separate Gold output instead of changing `fact_orders`.

Target

The regression target is `delivery_variance_days`:

- negative value = delivered earlier than estimated
- zero = delivered on the estimated date
- positive value = delivered late

Features

Only information known at or near order creation is used, avoiding target leakage. The final feature set includes:

- item count
- total item value
- total freight
- average product weight
- seller count
- product count
- promised delivery days
- purchase month
- purchase day of week

- customer state (indexed and one-hot encoded)

Post-delivery values such as actual shipping time or delivered timestamp are not used as predictors.

Model

The model is a PySpark ML `RandomForestRegressor` with 100 trees, maximum depth 8, and seed 42. Missing numeric features use median imputation; customer state is encoded before vector assembly.

Evaluation

Orders are split chronologically rather than randomly:

- training rows: **77,172** (80%)
- test rows: **19,298** (20%)
- chronological cutoff: approximately **2018-05-26 16:40:08**

Final Gold/table validation gives:

- Random Forest test MAE: approximately **5.05 days**
- median baseline MAE: **7.57 days**
- test R2: **0.530**
- MAE improvement versus baseline: approximately **33%**

The output table is:

```
olist.gold.order_delivery_predictions
```

with fields including `order_id`, `date_key`, actual delay, predicted delay, prediction error, absolute error, late flags, train/test split, and scoring timestamp.

Final table checks:

- rows: **96,470**
- distinct `order_id`: **96,470**
- null predictions: **0**

Power BI filters the ML validation page to `dataset_split = test` so the visuals show unseen holdout orders rather than training predictions.

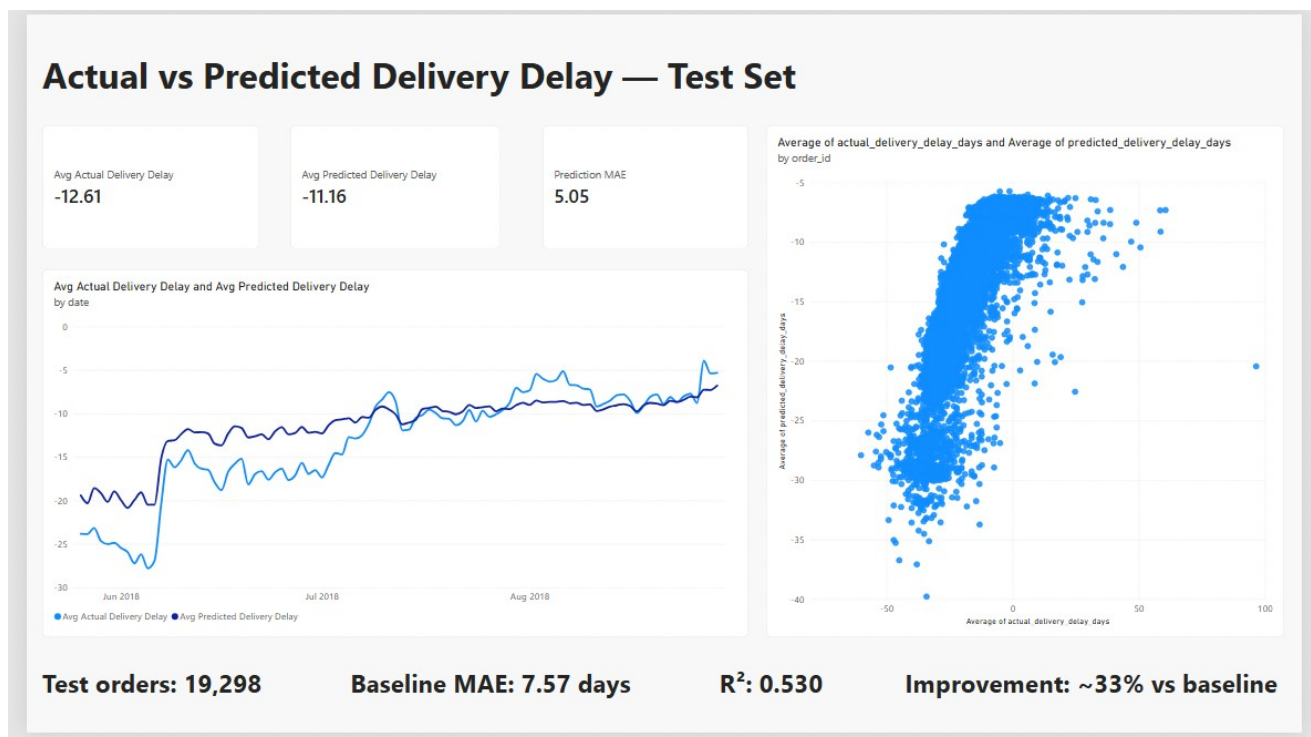


Figure 4. Power BI holdout-test view: actual versus predicted delivery delay and model validation metrics.

10. Challenges and lessons learned

Multiline review parsing. Row-count and score-column profiling exposed a source parsing issue that was not obvious from the file name alone. The lesson was to validate source behavior instead of trusting expected counts blindly.

Customer grain. Using `customer_id` initially creates an order-specific pseudo-dimension. Moving to `customer_unique_id` is necessary for correct customer-level analysis and repeat-customer measures.

Multi-fact modeling. Keeping order items and payments at separate grains is essential. A visually simple order-level join produces materially wrong revenue/payment totals.

Bundle path validation. Workspace notebooks can be referenced differently from repository files. Asset Bundle CI required the real `.ipynb` file extensions in local notebook references, which the validation workflow caught before deployment.

Lakehouse RLS versus report filters. RLS implemented in Unity Catalog protects DirectQuery data before Power BI receives it. This is stronger than only filtering visuals or relying on a report role.

ML leakage control. Delivery prediction is only defensible when the features are available before the outcome. Excluding handling/shipping/delivery-derived predictors and adding a baseline made the model evaluation meaningful.

Environment constraints. The project was built in a serverless Databricks Free Edition context and Power BI Desktop was used from Windows.

11. Reproducibility / runbook

10. Clone the repository.
11. Download the nine Olist CSV files from Kaggle.
12. Run `01-setup/00_bootstrap.sql`.
13. Upload the source files to `/Volumes/olist/landing/files/`.
14. Run the main Bronze -> Silver -> Gold -> Data Quality job.
15. Run the incremental demo if reproducing the watermark/MERGE behavior.
16. Run `07-performance/01_optimize` to reproduce the backend benchmark.
17. Run `databricks bundle validate --target dev` or push to GitHub to trigger CI bundle validation.
18. Optionally run `04-gold/10_rls_lakehouse.ipynb` to reproduce the lakehouse RLS demonstration.
19. Optionally run `04-gold/11_delivery_delay_predictions.ipynb` to rebuild the ML Gold output.
20. Open `powerbi/olist_report_import.pbix` for the primary report. Use `powerbi/olist_report_directquery.pbix` for the DirectQuery/RLS demonstration.

12. Submission summary

The submission provides a governed Databricks medallion pipeline, multi-fact dimensional model, automated DQ gate, incremental watermark/MERGE design, backend optimization evidence, Power BI semantic model/report, Git-based reproducibility, and advanced bonus work covering Auto Loader, Asset Bundle CI, lakehouse RLS, a DirectQuery variant, and ML delivery-delay scoring.

The only bonus item intentionally not completed in full is the formal Import-vs-DirectQuery performance comparison requested as part of Bonus 4. The implementation is documented exactly as delivered rather than overstated.